Corto 1

Ignacio Méndez Alvarez (22613) y Diego Soto Flores (22737)

Enlace al repositorio de GitHub: https://github.com/ignaciomendeza/VPC-CORTO1.git

Task 1

Inciso 1

Para la opción A, se tiene una W = 256, H = 256. Y para el Kernel K = 7, S = 2 y P = 0.

$$\textbf{Cálculo de } (W_{out}, H_{out})$$

$$\textbf{Fórmula general}$$

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

$$\textbf{Cálculo de } W_{out}$$

$$W_{out} = \left\lfloor \frac{256 - 7 + 2(0)}{2} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{249}{2} \right\rfloor + 1$$

$$W_{out} = 124 + 1$$

$$W_{out} = 125$$

$$\textbf{Cálculo de } H_{out}$$

$$H_{out} = \left\lfloor \frac{256 - 7 + 2(0)}{2} \right\rfloor + 1$$

$$H_{out} = 125$$

La salida de la opción A sería: W_{out} x H_{out} x C_{out}, es decir: 125 x 125 x 64

Para la opción B, se tiene una W = 256, H = 256. Y para el Kernel K = 3, S = 1 y P = 1.

Primera Convolución

$$\textbf{Cálculo de } W_{out}$$

$$W_{out} = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$W_{out} = \lfloor 255 \rfloor + 1$$

$$W_{out} = 256$$

$$\textbf{Cálculo de } H_{out}$$

$$H_{out} = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$H_{out} = 256$$

La salida de la primera convolución sería: W_{out} x H_{out}, es decir: 256 x 256

Segunda Convolución

$$\textbf{Cálculo de } W_{out}$$

$$W_{out} = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$W_{out} = \lfloor 255 \rfloor + 1$$

$$W_{out} = 256$$

$$\textbf{Cálculo de } H_{out}$$

$$H_{out} = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$H_{out} = 256$$

La salida de la segunda convolución sería: W_{out} x H_{out}, es decir: 256 x 256

Tercera Convolución

$$\textbf{Cálculo de } W_{out}$$

$$W_{out} = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$W_{out} = \lfloor 255 \rfloor + 1$$

$$W_{out} = 256$$

$$\textbf{Cálculo de } H_{out}$$

$$H_{out} = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$H_{out} = 256$$

La salida de la tercera convolución sería: W_{out} x H_{out}, es decir: 256 x 256

Capa de Max Pooling, se tiene K = 2, P = 0 y S = 2

$$\textbf{Cálculo de } W_{out}$$

$$W_{out} = \left\lfloor \frac{256 - 2}{2} \right\rfloor + 1$$

$$W_{out} = \lfloor 127 \rfloor + 1$$

$$W_{out} = 128$$

$$\textbf{Cálculo de } H_{out}$$

$$H_{out} = \left\lfloor \frac{256 - 2}{2} \right\rfloor + 1$$

$$H_{out} = \lfloor 127 \rfloor + 1$$

$$H_{out} = 128$$

La salida de la opción B sería: W_{out} x H_{out} x C_{out}, es decir: 128 x 128 x 64

Inciso 2

Para la opción A, se tiene K = 7, C_{in} = 4, C_{out} = 64

$$\textbf{Parámetros}$$

$$Parametros = K \times K \times C_{in} \times C_{out}$$

$$Parametros = 7 \times 7 \times 4 \times 64$$

$$Parametros = 49 \times 4 \times 64$$

$$Parametros = 196 \times 64$$

$$Parametros = 12544$$

En la opción A, serían 12,544 parámetros

Para la opción B, se tiene K = 3, C_{in} = 4, C_{out} = 64

$$Parametros = 3 \times 3 \times 4 \times 64$$

$$Parametros = 9 \times 4 \times 64$$

$$Parametros = 36 \times 4096$$

$$Parametros = 2304$$

Serían 2,304 parámetros en la convolución 1

$$Parametros = 3 \times 3 \times 64 \times 64$$
$$Parametros = 9 \times 64 \times 64$$
$$Parametros = 9 \times 4096$$
$$Parametros = 36864$$

Serían 36,864 parámetros en la convolución 2

$$Parametros = 3 \times 3 \times 64 \times 64$$
$$Parametros = 9 \times 64 \times 64$$
$$Parametros = 9 \times 4096$$
$$Parametros = 36864$$

Serían 36,864 parámetros en la convolución 3

Para la opción B, en total por las 3 convoluciones serían 76,032 parámetros

La Opción B se estandarizó porque varias convoluciones pequeñas (3×3) pueden lograr un campo receptivo equivalente a filtros más grandes, pero introduciendo más capas no lineales (ReLU) entre ellas. Esto permite aprender representaciones más complejas. Además, los kernels pequeños reducen el número de parámetros y mejoran la eficiencia computacional, manteniendo mejor la estructura espacial de la imagen. Este diseño se hizo bastante popular por arquitecturas como VGGNet, que fue propuesta por Karen Simonyan y Andrew Zisserman.

Referencia: Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. ICLR.

Task 2

Inciso 1

Los métodos tradicionales como SIFT y HOG dependen de descriptores de características diseñados manualmente que encuentran patrones locales de bordes o gradientes. Aunque pueden funcionar en entornos controlados, en un contexto agrícola real pueden ser limitantes, ya que las hojas pueden aparecer con condiciones distintas de iluminación, diferentes ángulos en los que se tomó la foto, o tapadas en parte por otras hojas o ramas. Estas variaciones cambian bastante los gradientes y puntos clave que estos algoritmos usan, lo que hace que haya una pérdida de consistencia en los descriptores y disminuye la capacidad de un clasificador como un SVM para identificar correctamente las enfermedades.

Por otro lado, los modelos de Deep Learning basados en redes convolucionales generan mapas de características jerárquicos que aprenden automáticamente representaciones

visuales desde patrones simples hasta estructuras semánticas más complejas. En las primeras capas se encuentran bordes y texturas, mientras que en capas profundas se capturan patrones de alto nivel relacionados con la forma y las manchas que caracterizan una enfermedad. Esta representación jerárquica hace que el modelo sea mucho más completo con las variaciones de iluminación y perspectiva, ya que las características importantes se aprenden directamente de los datos en lugar de que sean definidas manualmente.

Inciso 2

Si tuviera que elegir entre AlexNet y VGG-16, elegiría VGG-16 porque su arquitectura es más profunda y modular, y está basada en múltiples convoluciones pequeñas de 3×3. Esta estructura permite aumentar poco a poco el campo receptivo y al mismo tiempo mantiene una organización consistente de bloques convolucionales, lo que hace más fácil el uso de transfer learning para reutilizar características aprendidas en grandes datasets. Además, la presencia de múltiples funciones de activación ReLU introduce más no-linealidad en la red, aumentando su capacidad para aprender representaciones visuales complejas y abstractas. Esto es útil para identificar patrones de enfermedades en hojas de cultivo.

Task 3

In [ ]:
```
import kagglehub
import os
import random
import numpy as np
import torch
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import os
import torch
import torch.nn as nn
from torchvision import models
from tqdm import tqdm
import copy
import numpy as np
import torch
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import numpy as np
```

In [ ]:
```
path = kagglehub.dataset_download("aryashah2k/mango-leaf-disease-dataset")

print("Path to dataset files:", path)
```

```
Downloading to C:\Users\dfsf2\.cache\kagglehub\datasets\aryashah2k\mango-leaf-diseas
e-dataset\1.archive...
100%|██████████| 103M/103M [00:09<00:00, 10.9MB/s]
Extracting files...
```

```
Path to dataset files: C:\Users\dfsf2\.cache\kagglehub\datasets\aryashah2k\mango-lea
f-disease-dataset\versions\1
```

In [ ]:
```
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Dispositivo:", device)

data_dir = r"C:\Users\dfsf2\.cache\kagglehub\datasets\aryashah2k\mango-leaf-disease
```

```
Dispositivo: cpu
```

In [ ]:
```
# Transformaciones para entrenamiento
transform_train = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# Transformaciones para validación
transform_val = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# Cargar dataset
dataset = datasets.ImageFolder(data_dir)

# Separar entrenamiento y validación (80% / 20%)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size

train_dataset, val_dataset = torch.utils.data.random_split(dataset, [train_size, va

# Aplicar transformaciones
train_dataset.dataset.transform = transform_train
val_dataset.dataset.transform = transform_val

# DataLoaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

print("Clases:", dataset.classes)
print("Imagenes entrenamiento:", train_size)
print("Imagenes validación:", val_size)
```

```
Clases: ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Gall Midg
e', 'Healthy', 'Powdery Mildew', 'Sooty Mould']
Imagenes entrenamiento: 3200
Imagenes validación: 800
```

In [ ]:
```
num_clases = len(dataset.classes)

# Cargar VGG16 preentrenada
modelo = models.vgg16(pretrained=True)

# Congelar todas las capas convolucionales
for param in modelo.features.parameters():
    param.requires_grad = False

# VGG16
modelo.classifier[6] = nn.Linear(in_features=4096, out_features=num_clases)

modelo = modelo.to(device)

entrenables = sum(p.requires_grad for p in modelo.parameters())
totales = sum(1 for _ in modelo.parameters())
print(f"Parámetros entrenables: {entrenables} / {totales}")
print("Última capa:", modelo.classifier[6])
```

```
c:\Users\dfsf2\anaconda3\Lib\site-packages\torchvision\models\_utils.py:208: UserWar
ning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the
future, please use 'weights' instead.
  warnings.warn(
c:\Users\dfsf2\anaconda3\Lib\site-packages\torchvision\models\_utils.py:223: UserWar
ning: Arguments other than a weight enum or `None` for 'weights' are deprecated sinc
e 0.13 and may be removed in the future. The current behavior is equivalent to passi
ng `weights=VGG16_Weights.IMAGENET1K_V1`. You can also use `weights=VGG16_Weights.DE
FAULT` to get the most up-to-date weights.
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to C:\Users\df
sf2/.cache\torch\hub\checkpoints\vgg16-397923af.pth
100%|██████████| 528M/528M [00:48<00:00, 11.4MB/s]
Parámetros entrenables: 6 / 32
Última capa: Linear(in_features=4096, out_features=8, bias=True)
```

In [ ]:
```
def entrenar_una_epoca(modelo, loader, criterio, optimizador, device):
    modelo.train()
    perdida_total = 0.0
    correctas = 0
    total = 0

    barra = tqdm(loader, desc="Entrenando", leave=False)

    for x, y in barra:
        x, y = x.to(device), y.to(device)

        optimizador.zero_grad()
        logits = modelo(x)
        perdida = criterio(logits, y)
        perdida.backward()
        optimizador.step()
```

```
        perdida_total += perdida.item() * x.size(0)
        preds = logits.argmax(dim=1)
        correctas += (preds == y).sum().item()
        total += y.size(0)

        barra.set_postfix(loss=perdida.item())

    return perdida_total / total, correctas / total


@torch.no_grad()
def evaluar(modelo, loader, criterio, device):
    modelo.eval()
    perdida_total = 0.0
    correctas = 0
    total = 0

    barra = tqdm(loader, desc="Validando", leave=False)

    for x, y in barra:
        x, y = x.to(device), y.to(device)

        logits = modelo(x)
        perdida = criterio(logits, y)

        perdida_total += perdida.item() * x.size(0)
        preds = logits.argmax(dim=1)
        correctas += (preds == y).sum().item()
        total += y.size(0)

    return perdida_total / total, correctas / total
```

In [ ]:

```
epocas = 5

mejor_acc = 0.0
mejor_pesos = copy.deepcopy(modelo.state_dict())

historial = {
    "train_loss": [],
    "train_acc": [],
    "val_loss": [],
    "val_acc": []
}

for epoca in range(1, epocas + 1):
    train_loss, train_acc = entrenar_una_epoca(modelo, train_loader, criterio, opti
    val_loss, val_acc = evaluar(modelo, val_loader, criterio, device)

    historial["train_loss"].append(train_loss)
    historial["train_acc"].append(train_acc)
    historial["val_loss"].append(val_loss)
    historial["val_acc"].append(val_acc)

    print(f"Época {epoca}/{epocas} | "
          f"train_loss={train_loss:.4f}, train_acc={train_acc:.4f} | "
```

```
            f"val_loss={val_loss:.4f}, val_acc={val_acc:.4f}")

    if val_acc > mejor_acc:
        mejor_acc = val_acc
        mejor_pesos = copy.deepcopy(modelo.state_dict())

# Cargar el mejor modelo
modelo.load_state_dict(mejor_pesos)
print("Mejor val_acc:", mejor_acc)
```

```
Época 1/5 | train_loss=0.1028, train_acc=0.9731 | val_loss=0.0587, val_acc=0.9900

Época 2/5 | train_loss=0.0683, train_acc=0.9822 | val_loss=0.0503, val_acc=0.9888

Época 3/5 | train_loss=0.0567, train_acc=0.9866 | val_loss=0.0371, val_acc=0.9912

Época 4/5 | train_loss=0.0516, train_acc=0.9859 | val_loss=0.0400, val_acc=0.9912

Época 5/5 | train_loss=0.0362, train_acc=0.9928 | val_loss=0.0345, val_acc=0.9888
Mejor val_acc: 0.99125
```

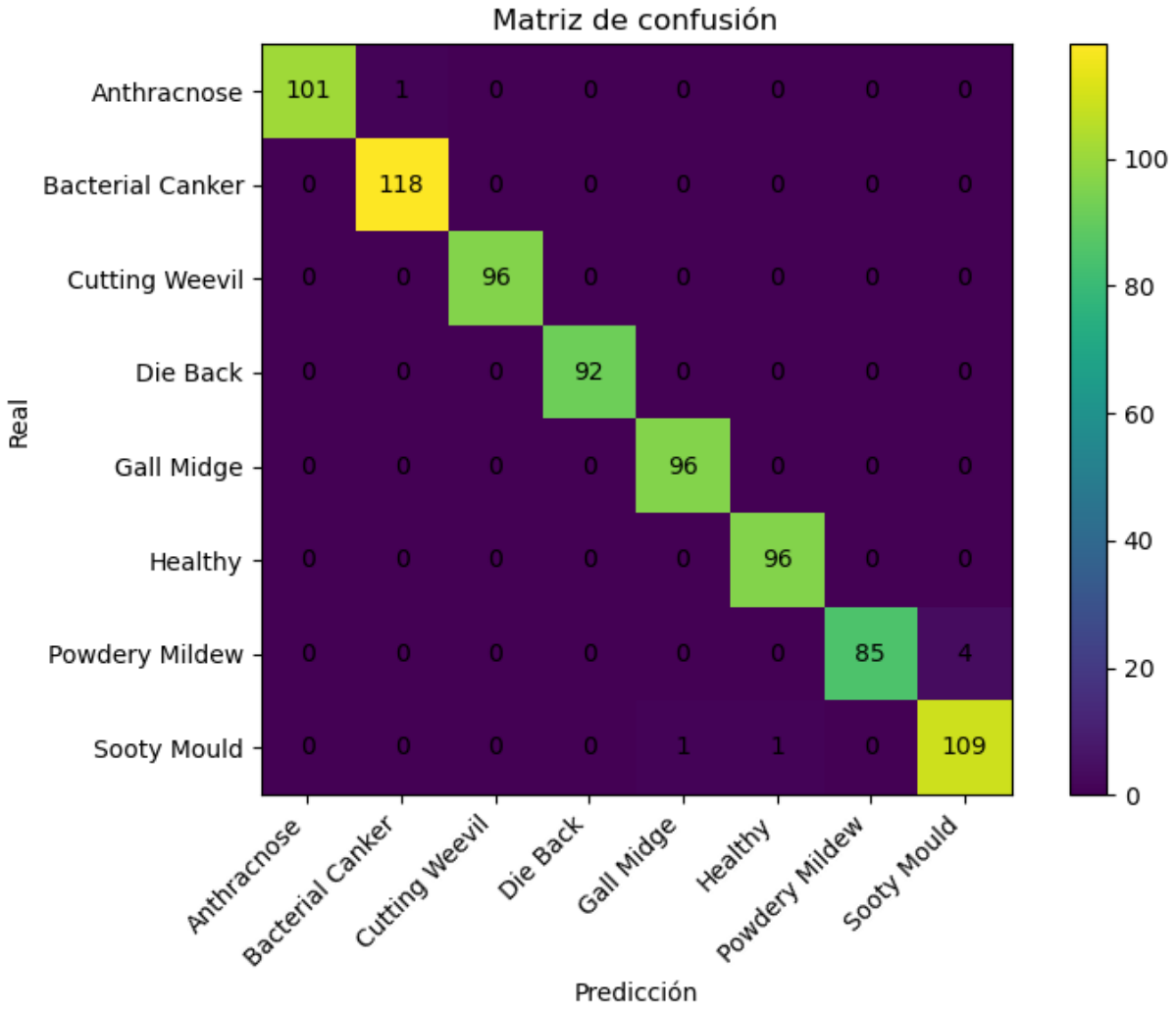
In [ ]:

```
@torch.no_grad()
def obtener_preds_y_true(modelo, loader, device):
    modelo.eval()
    y_true = []
    y_pred = []

    for x, y in loader:
        x = x.to(device)
        logits = modelo(x)
        preds = logits.argmax(dim=1).cpu().numpy()

        y_true.extend(y.numpy())
        y_pred.extend(preds)

    return np.array(y_true), np.array(y_pred)

y_true, y_pred = obtener_preds_y_true(modelo, val_loader, device)

acc_final = accuracy_score(y_true, y_pred)
cm = confusion_matrix(y_true, y_pred)

print("Accuracy en validación:", acc_final)
print("\nMatriz de confusión:\n", cm)
print("\nOrden de clases:", dataset.classes)
```

```
Accuracy en validación: 0.99125

Matriz de confusión:
 [[101   1   0   0   0   0   0   0]
 [  0 118   0   0   0   0   0   0]
 [  0   0  96   0   0   0   0   0]
 [  0   0   0  92   0   0   0   0]
 [  0   0   0   0  96   0   0   0]
 [  0   0   0   0   0  96   0   0]
 [  0   0   0   0   0   0  85   4]
 [  0   0   0   0   1   1   0 109]]

Orden de clases: ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', '
Gall Midge', 'Healthy', 'Powdery Mildew', 'Sooty Mould']
```

In [ ]:

```python
def plot_matriz_confusion(cm, clases, titulo="Matriz de confusión"):
    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(cm)

    ax.set_title(titulo)
    ax.set_xlabel("Predicción")
    ax.set_ylabel("Real")

    ax.set_xticks(np.arange(len(clases)))
    ax.set_yticks(np.arange(len(clases)))
    ax.set_xticklabels(clases, rotation=45, ha="right")
    ax.set_yticklabels(clases)

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, str(cm[i, j]), ha="center", va="center")

    fig.colorbar(im, ax=ax)
    plt.tight_layout()
    plt.show()

plot_matriz_confusion(cm, dataset.classes)
```



La red obtuvo un accuracy del 0.99 en validación. Con estos resultados obtenidos se puede apreciar que no hay overfitting significativo, ya que el accuracy en validación es alta y consistente con el accuracy del entrenamiento indicando que el modelo tiene un buen desempeño. Esto indica que la red VGG16 logro generalizar correctamente y no memorizo las imagenes del entrenamiento. En este caso no se aplico un Dropout manual, pero la red VGG16 ya incluye uno en su arquitectura.